

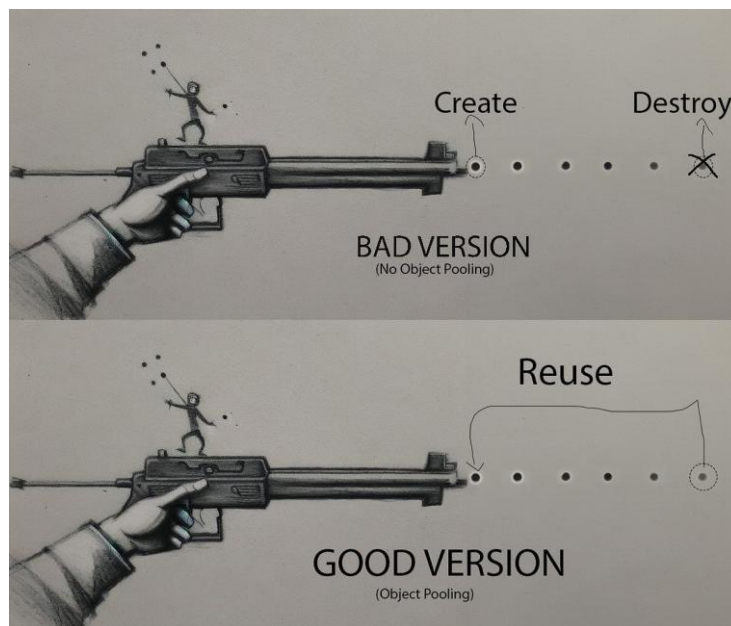


LẬP TRÌNH GAME 3D NÂNG CAO

BÀI 7 : Object Pooling: Tối ưu hóa hiệu suất
trong UNITY 3D

- ☐ Object Pooling
- ☐ Cách sử dụng Object Pooling

❑ **Object Pooling** : một kỹ thuật quản lý bộ nhớ phổ biến trong Unity, dùng để tái sử dụng các đối tượng (objects) thay vì tạo và hủy chúng liên tục, giúp tăng hiệu suất cho game.



- ❑ Mục tiêu
- ❑ Giảm chi phí hiệu suất khi tạo hoặc hủy đối tượng.
- ❑ Tăng tốc độ xử lý trong các game yêu cầu hiệu suất cao (như bắn súng, game đua xe).

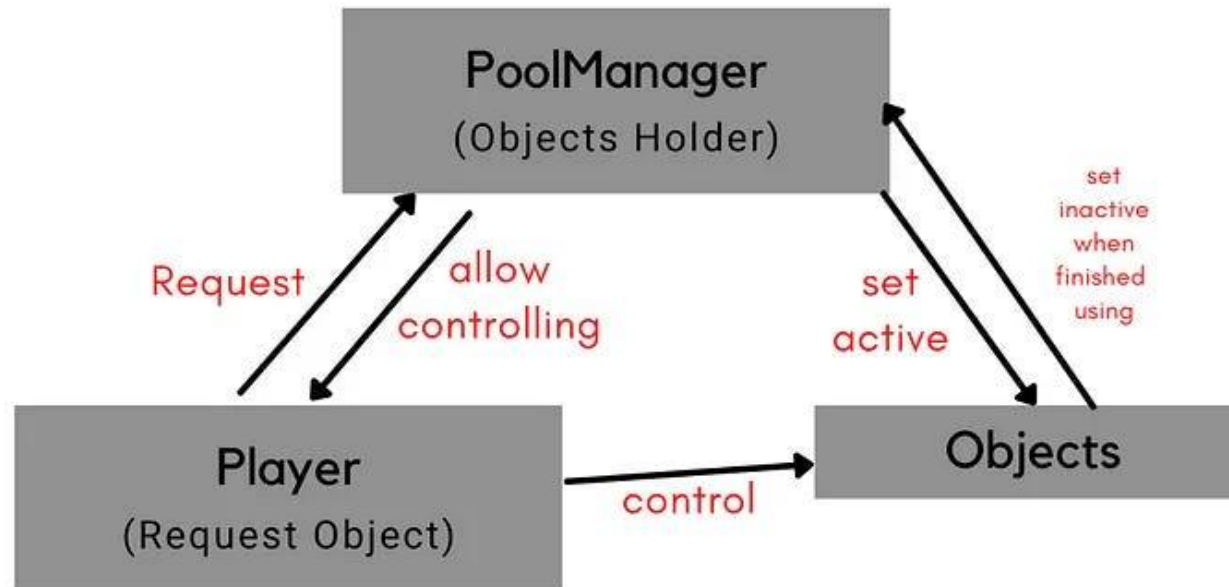
- ❑ Mục tiêu
- ❑ Giảm chi phí hiệu suất khi tạo hoặc hủy đối tượng.
- ❑ Tăng tốc độ xử lý trong các game yêu cầu hiệu suất cao (như bắn súng, game đua xe).

- ❑ Cách hoạt động
- ❑ Tạo sẵn một danh sách (pool) các đối tượng trong bộ nhớ khi bắt đầu game.
- ❑ Khi cần dùng đối tượng:
- ❑ Lấy từ danh sách pool (thay vì tạo mới).
- ❑ Khi không còn sử dụng -> Trả lại vào pool (thay vì hủy).

- ❑ Cách hoạt động
- ❑ Tạo sẵn một danh sách (pool) các đối tượng trong bộ nhớ khi bắt đầu game.
- ❑ Khi cần dùng đối tượng: Lấy từ danh sách pool (thay vì tạo mới).
- ❑ Khi không còn sử dụng -> Trả lại vào pool (thay vì hủy).

□ Mô hình :

Object Pooling Design Pattern



- ☐ Ưu điểm
- ☐ Giảm tải công việc của Garbage Collector.
- ☐ Tăng hiệu suất cho các game có số lượng đối tượng lớn.

- ❑ Nhược điểm
- ❑ Tốn bộ nhớ: Giữ các đối tượng không dùng trong bộ nhớ gây lãng phí.
- ❑ Phức tạp hơn và tăng thời gian khởi động Game
- ❑ Kích thước pool khó cân đối: Quá lớn lãng phí, quá nhỏ lại không đủ dùng.

□ Cách khai báo ObjectPool

```
4 public class ObjectPool : MonoBehaviour
5 {
6     public static ObjectPool SharedInstance; // Singleton để truy cập pool từ bất kỳ đâu
7     public List<GameObject> pooledObjects; // Danh sách lưu các đối tượng trong pool * Serializable
8     public GameObject objectToPool; // Prefab cần quản lý * Unchanged
9     public int amountToPool; // Số lượng đối tượng cần tạo trước * Unchanged
10
11     void Awake()
12     {
13         SharedInstance = this;
14     }
```

❑ Khởi tạo sẵn các GameObject và lưu vào **Pool**

```
15 void Start()
16 {
17     // Khởi tạo trước 1 danh sách GameObject,
18     // đặt trạng thái Active False
19     pooledObjects = new List<GameObject>();
20     for (int i = 0; i < amountToPool; i++)
21     {
22         GameObject obj = Instantiate(objectToPool);
23         obj.SetActive(false);
24         pooledObjects.Add(obj);
25     }
26 }
```

❑ Hàm Get Object ở trạng thái Active == False

```
? usages ? overrides ? extension methods ? exposing APIs
27 public GameObject GetPooledObject()
28 {
29     for (int i = 0; i < amountToPool; i++)
30     {
31         if (!pooledObjects[i].activeInHierarchy)
32         {
33             return pooledObjects[i];
34         }
35     }
36     return null; // Trả về null nếu không có đối tượng nào khả dùng
37 }
```

❑ Hàm Get Object ở trạng thái Active == False

```
? usages ? overrides ? extension methods ? exposing APIs
27 public GameObject GetPooledObject()
28 {
29     for (int i = 0; i < amountToPool; i++)
30     {
31         if (!pooledObjects[i].activeInHierarchy)
32         {
33             return pooledObjects[i];
34         }
35     }
36     return null; // Trả về null nếu không có đối tượng nào khả dùng
37 }
```

□ Class gọi GameObject từ Pool

```
3 public class TurretShooting : MonoBehaviour
4 {
5     public Transform turret; * Unchanged
6     ? usages ? overrides ? extension methods ? exposing APIs
7     void Shoot()
8     {
9         // Lấy đối tượng từ Object Pool
10        GameObject bullet = ObjectPool.SharedInstance.GetPooledObject();
11        if (bullet != null)
12        {
13            bullet.transform.position = turret.position;
14            bullet.transform.rotation = turret.rotation;
15            bullet.SetActive(true); // Kích hoạt đối tượng
16        }
17    }
18    ? usages ? overrides ? extension methods ? exposing APIs
19    void OnBulletHit(GameObject bullet)
20    {
21        bullet.SetActive(false); // Trả lại đối tượng vào pool
22    }
23 }
```

- ☐ Object Pooling
- ☐ Cách sử dụng Object Pooling
- ☐ Thực hành



LẬP TRÌNH GAME 3D NÂNG CAO

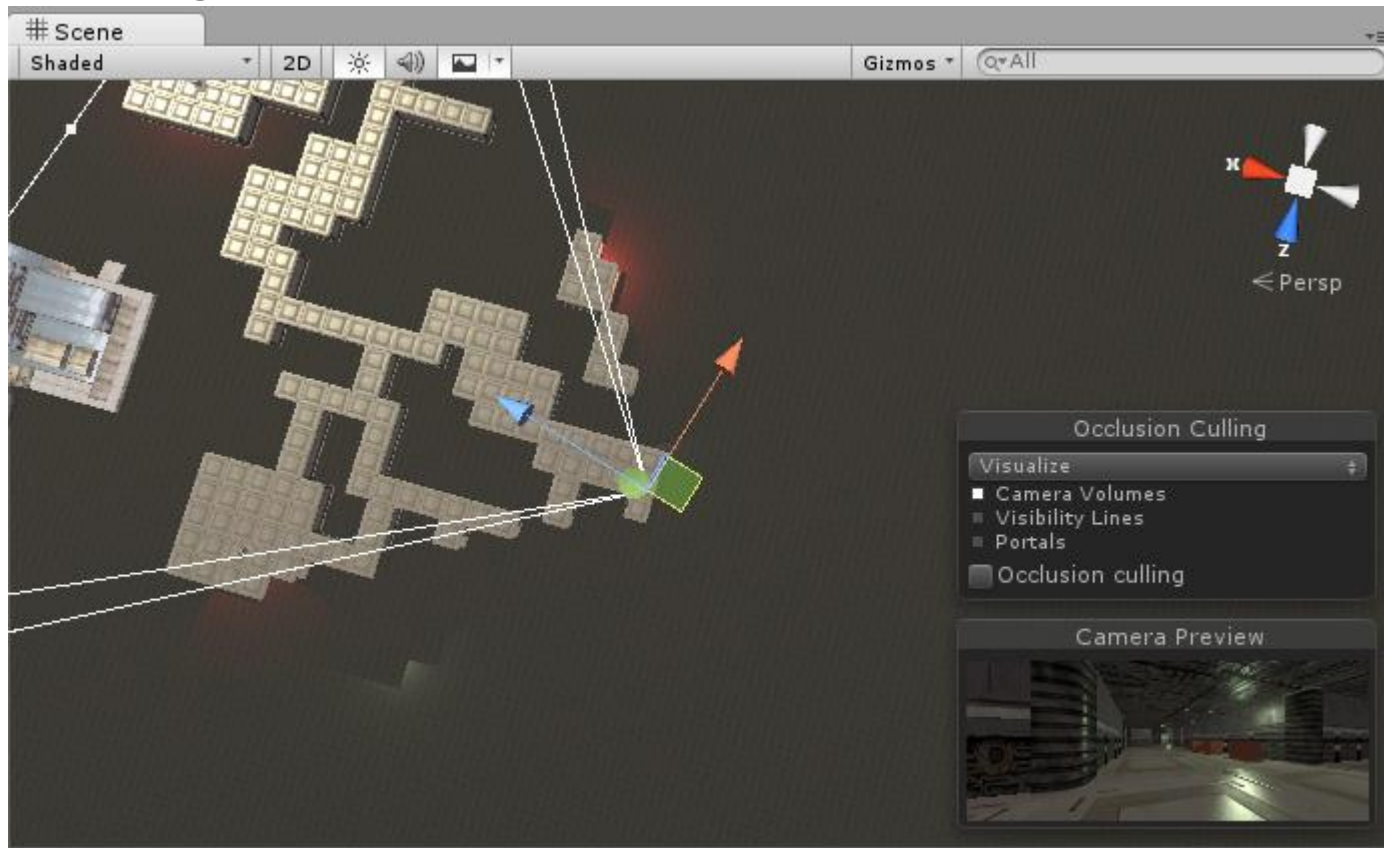
BÀI 7: Occlusion Culling và LOD trong Unity 3D

☐ Occlusion Culling

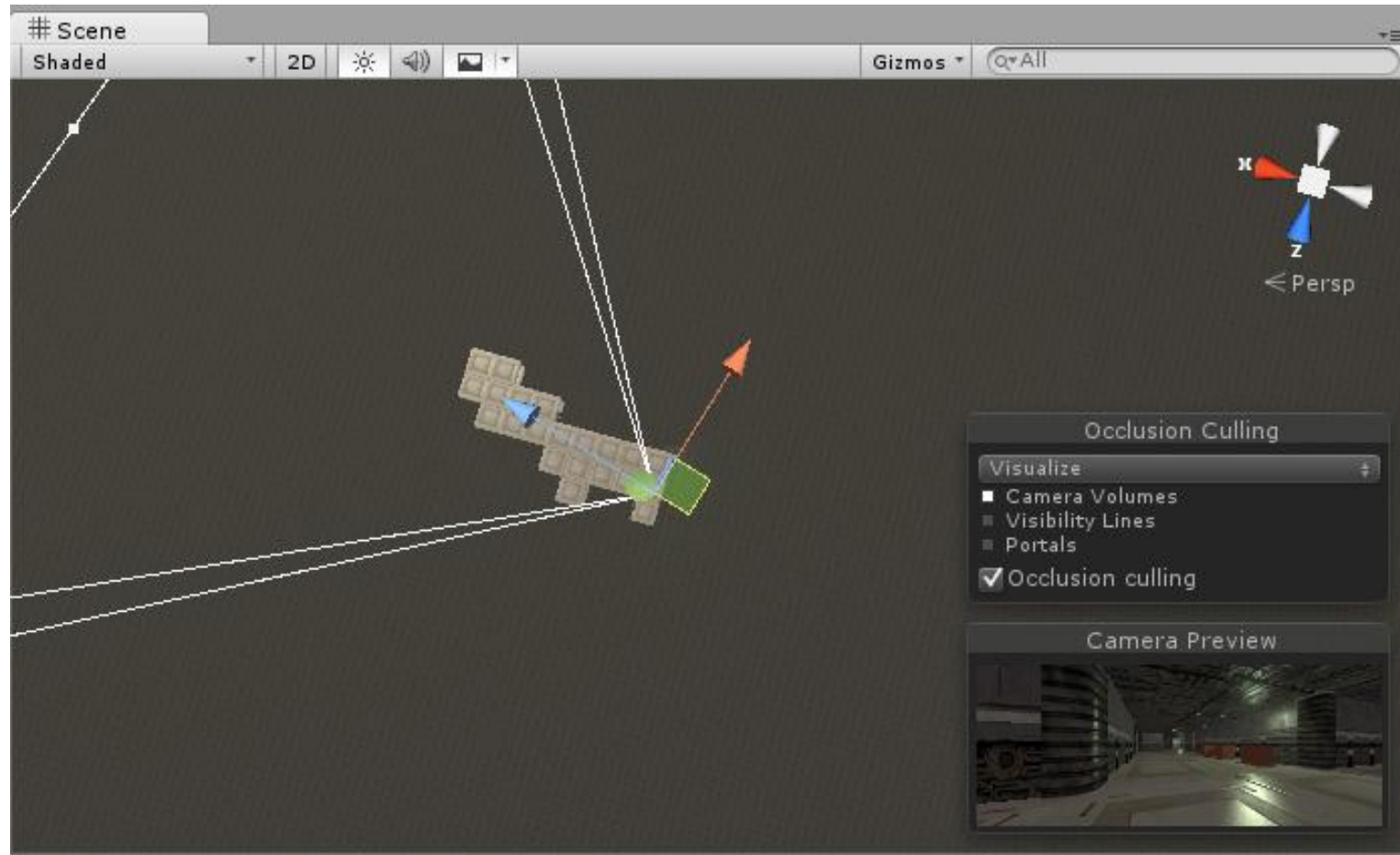
☐ Cách sử dụng

□ **Occlusion Culling** : là một kỹ thuật trong Unity dùng để ngăn chặn việc tính toán và hiển thị các GameObject bị che khuất hoàn toàn (occluded) bởi các đối tượng khác. Điều này giúp tiết kiệm tài nguyên CPU và GPU, cải thiện hiệu suất game.

□ Cách Camera render trong tình huống thông thường



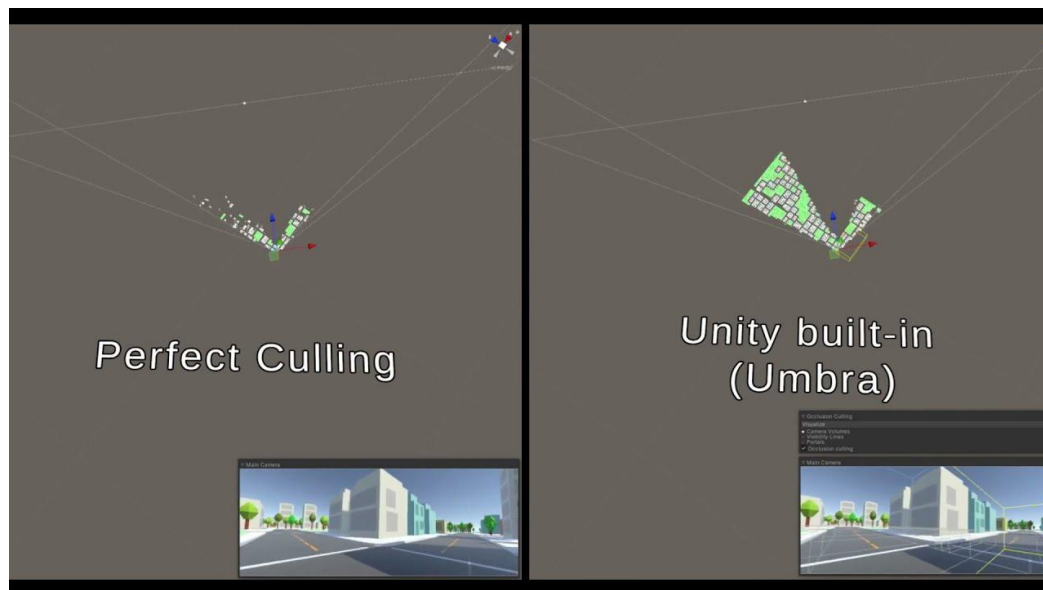
□ Sau khi áp dụng Occlusion Culling



- ❑ Khi nào nên sử dụng Occlusion Culling?
- ❑ Game có môi trường phức tạp:
- ❑ Nhiều khu vực nhỏ, tách biệt bởi các vật thể lớn như phòng hoặc hành lang.

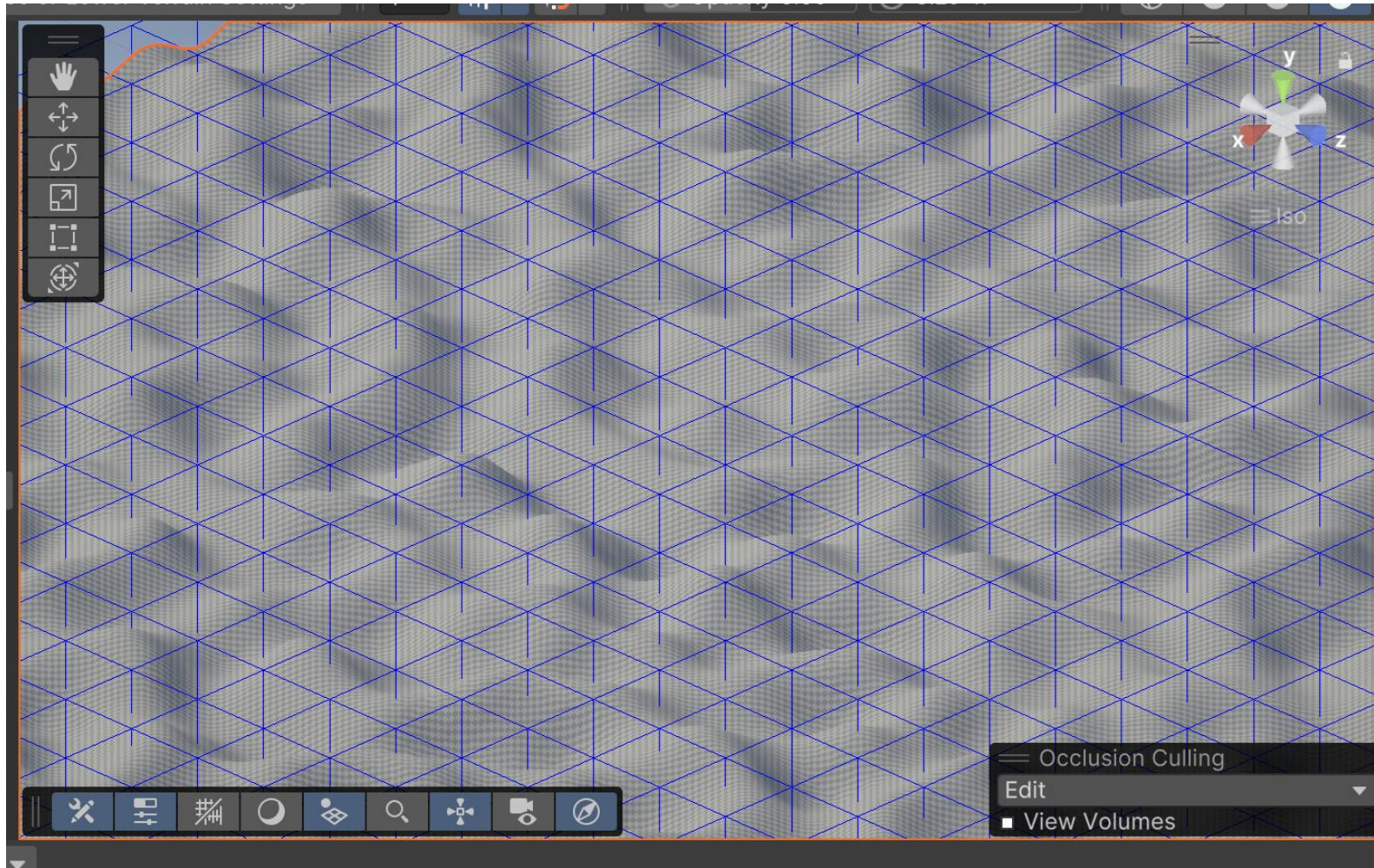


- ❑ Dự án bị giới hạn hiệu suất GPU (GPU-bound):
- ❑ Khi có quá nhiều đối tượng được vẽ, Occlusion Culling giảm tải GPU bằng cách không tính toán các đối tượng bị che khuất.



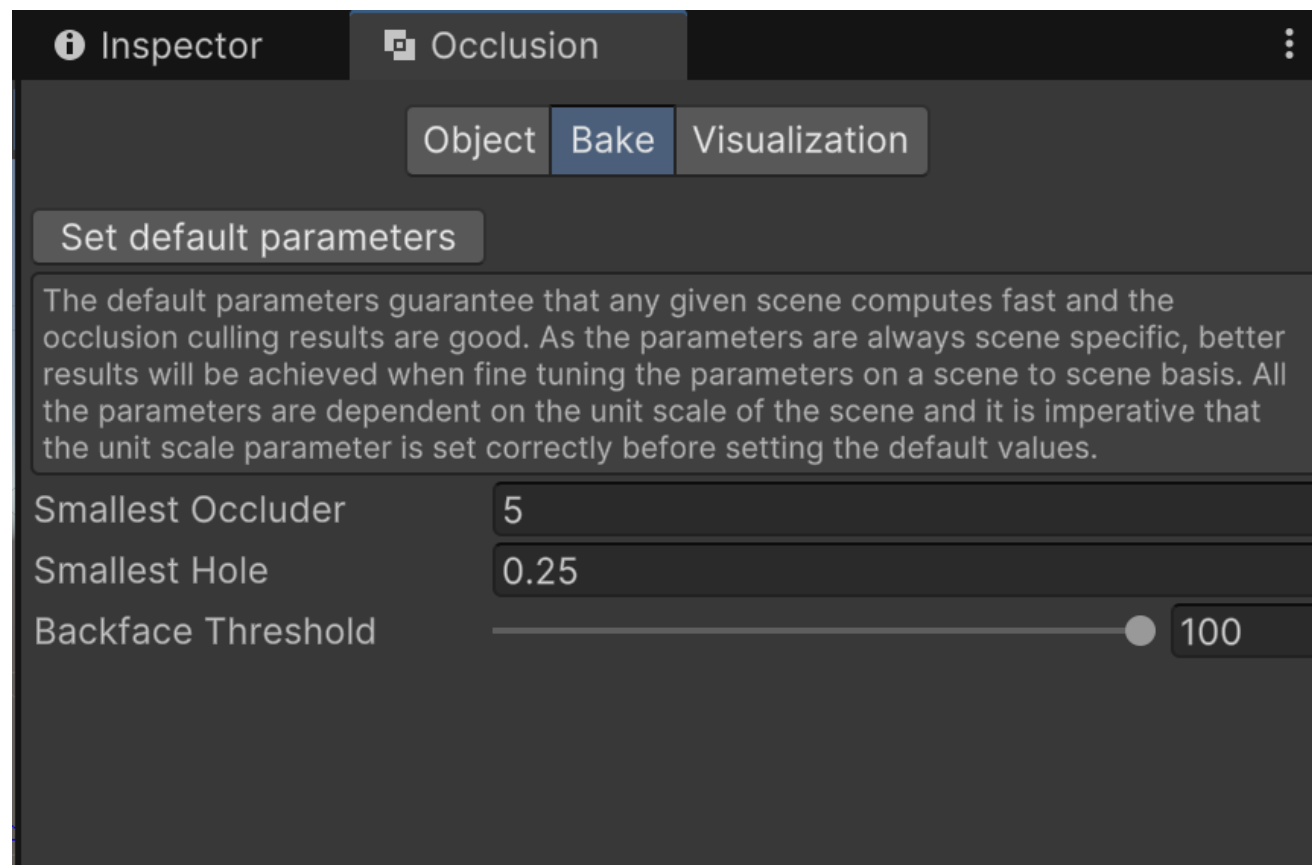
- ❑ “**Baking**” dữ liệu: Unity chia Scene thành các ô (cells) và phân tích dữ liệu hiển thị giữa các ô này.
- ❑ Dữ liệu được nén để giảm kích thước nhưng vẫn đảm bảo độ chính xác.
- ❑ Runtime: Unity tải dữ liệu đã “**bake**” vào bộ nhớ.
- ❑ Camera kiểm tra dữ liệu để xác định những gì cần hiển thị và những gì cần loại bỏ.

□ Chia các Cell để tính toán



□ Truy cập Windows -> Rendering -> Occlusion

Culling. Bấm nút Bake để Unity tính toán



- ❑ Kích thước tối thiểu của một vật thể được coi là "vật cản" (Occluder).
- ❑ Tác dụng: Các vật thể nhỏ hơn giá trị này sẽ không được xem là vật cản che khuất tầm nhìn.
- ❑ Cách điều chỉnh:
- ❑ Giá trị lớn: Loại bỏ các vật thể nhỏ, chỉ giữ lại các vật cản lớn (như tòa nhà, tường).
- ❑ Giá trị nhỏ: Bao gồm cả các vật thể nhỏ như cây cối, đồ nội thất

- ❑ Ý nghĩa: Kích thước “Hole” nhỏ nhất mà qua đó camera có thể "nhìn xuyên qua".
- ❑ Tác dụng: Các “Hole” nhỏ hơn giá trị này sẽ bị xem như bị che khuất hoàn toàn.
- ❑ Giá trị lớn: Loại bỏ các “Hole” nhỏ, chỉ giữ lại các khu vực mở rộng lớn.
- ❑ Giá trị nhỏ: Bao gồm cả các chi tiết nhỏ hơn, cho phép nhìn qua các “Hole” nhỏ.

- ❑ Phần trăm diện tích bề mặt quay mặt ra xa camera được tính là "mặt sau" (Backface).
- ❑ Tác dụng: Xác định mức độ mà mặt sau của một đối tượng có thể bị loại bỏ khỏi tính toán hiển thị.
- ❑ Giá trị cao (gần 100): Bao gồm nhiều bề mặt quay xa camera hơn.
- ❑ Giá trị thấp: Loại bỏ nhiều bề mặt không nhìn thấy khỏi tính toán.

- ❑ Phần trăm diện tích bề mặt quay mặt ra xa camera được tính là "mặt sau" (Backface).
- ❑ Tác dụng: Xác định mức độ mà mặt sau của một đối tượng có thể bị loại bỏ khỏi tính toán hiển thị.
- ❑ Giá trị cao (gần 100): Bao gồm nhiều bề mặt quay xa camera hơn.
- ❑ Giá trị thấp: Loại bỏ nhiều bề mặt không nhìn thấy khỏi tính toán.

- ☐ Occlusion Culling (OC)
- ☐ Thực hành sử dụng OC



Kết thúc